1. A knight enters a castle containing `n` rooms arranged in a line.

Each room contains:

gold[i]

The knight wants to collect maximum gold.

But every room has a security level:

security[i]

The knight can enter or skip a room.

Rules

1. The knight cannot enter two consecutive rooms.
2. If he enters room `i`, he must skip the next room.
3. He can enter at most `k` rooms.
4. The total security level of all entered rooms cannot exceed `S`.
5. If a room contains more than `X` gold, entering it forces the knight to skip **two additional rooms** instead of one.

Find the maximum gold that can be collected.

Test case-1:

gold = [10, 20, 30]
security = [1, 2, 3]

k = 2
S = 10
X = 100

Test case-2:

gold = [20, 100, 50]
security = [2, 3, 2]

k = 2
S = 10
X = 80

Test case-3:

gold   = [40, 100, 50, 60]
security = [1,   4, 2, 2]

k = 2
S = 3
X = 80

Solution:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int fn(int i,
       vector<int>& gold,
       vector<int>& security,
       int k,
       int S,
       int roomsTaken,
       int securityUsed) {

    // Base case:
    // No more rooms
    if (i >= gold.size()) {
        return 0;
    }

    // Cannot enter more than k rooms
```

```
if (roomsTaken == k) {
    return 0;
}

// ------------------------
// Option 1: Skip room i
// ------------------------
int skip = fn(
    i + 1,
    gold,
    security,
    k,
    S,
    roomsTaken,
    securityUsed
);


// ------------------------
// Option 2: Enter room i
// ------------------------

int take = 0;

// Check security constraint
if (securityUsed + security[i] <= S) {

    // Determine how many rooms to skip
    int nextIndex;

    if (gold[i] > 80) {
        // Skip next TWO additional rooms
        nextIndex = i + 3;
    }
    else {
        // Skip only the next room
        nextIndex = i + 2;
```

```
        }

        take = gold[i] + fn(
            nextIndex,
            gold,
            security,
            k,
            S,
            roomsTaken + 1,
            securityUsed + security[i]
        );
    }

    return max(skip, take);
}


int main() {

    vector<int> gold =
        {20, 50, 30, 100, 40, 80, 60};

    vector<int> security =
        {2, 3, 1, 5, 2, 4, 3};

    int k = 3;
    int S = 10;
    int X = 80;

    cout << fn(
        0,
        gold,
        security,
        k,
        S,
        0,
        0
```

```
    );

    return 0;
}
```

****

2. Rama started on a quest of visiting temples, with few coins in his pockets.

- As soon as he visits any temple, the money in his pocket gets doubled, and on his way out, he donates Rs. 100 to each temple.
- He visits 4 temples on his quest. After visiting the last temple, his pocket is empty of all the money, so he doesn't have the money he had initially.

  Solution:

1st Temple:

As soon as he enters the temple, his money is doubled to 2x. On his way out he donates Rs. 100. Therefore, he is left with Rs. 2x - 100.

2nd Temple:

As soon as he enters the temple, his money is doubled to 2(2x - 100). On his way out he donates another Rs. 100 and he is left out with Rs. (4x - 300).

3rd Temple:

As soon as he enters the temple, his money is doubled to 2(4x - 300). On his way out he donates another Rs. 100 and he is left with Rs. (8x - 700).

4th Temple:

As soon as he enters the temple, his money is doubled to 2(8x - 700). On his way out he donates another Rs. 100 and he is left with Rs. (16x - 1500). According to the question, after his last visit, he completely runs out of money.

```
Therefore,
=> 16x - 1500 = 0;
=> x = 1500/16;
=> x = 93.75
```

Rama had started his quest with **Rs 93.75** in his pocket.

3. There are 25 horses among which you need to find out the fastest 3 horses. You can conduct a race among at most 5 to find out their relative speed. At no point can you find out the actual speed of the horse in a race. Find out the minimum no. of races which are required to get the top 3 horses.